Data Structures and Algorithms
Assignment 2: Binary Trees and Binary Search Trees

Program templates for questions 1-4 are given as separated files. You must use them to implement your functions. You need to submit your code to the **https://www.hackerearth.com** (Email invitation will be sent to your school account). Deadline for program submission: [**27/09/2026**].

1. Write an **iterative** Python function inorderIterative() that prints the in-order traversal of a binary search tree using only one temporary stack inside the inorderIterative()function. Note that you should only use push() or pop() operations when you add or remove integers from the stack.

   The function prototype is given as follows:

   ```
   def inorderIterative(root: BSTNode):
   ```

   Let us consider the below tree for example

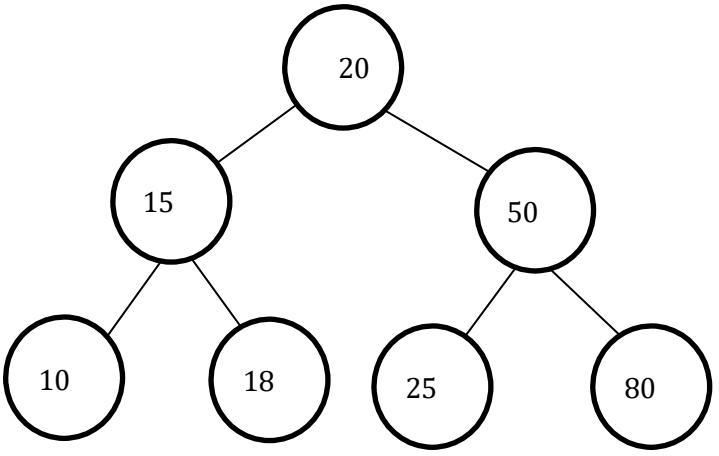


Figure 1: Iterative in-Order Traversal: **10 15 18 20 25 50 80**

Some sample inputs and outputs are given as follows:

```
1: Insert an integer into the binary search tree
2: Print the post-order traversal of the binary search tree
0: Quit

Please input your choice(1/2/0): 1
Input an integer to insert: 20

Please input your choice(1/2/0): 1
Input an integer to insert: 15

Please input your choice(1/2/0): 1
Input an integer to insert: 50

Please input your choice(1/2/0): 1
Input an integer to insert: 10

Please input your choice(1/2/0): 1
```

```
Input an integer to insert: 18

Please input your choice(1/2/0): 1
Input an integer to insert: 25

Please input your choice(1/2/0): 1
Input an integer to insert: 80

Please input your choice(1/2/0): 2
In-order traversal: 10 15 18 20 25 50 80

Please input your choice(1/2/0): 0
```

2. **(sumGreaterTree)** Given a Binary Search Tree (assume that you always receive a valid BST), write a function, `sumGreaterTree()` to transform the BST such that each node's value is replaced by the sum of all values of the nodes greater than or equal to that node (including itself).

The function prototype is given as follows:

```
def sumGreaterTree(self):
```

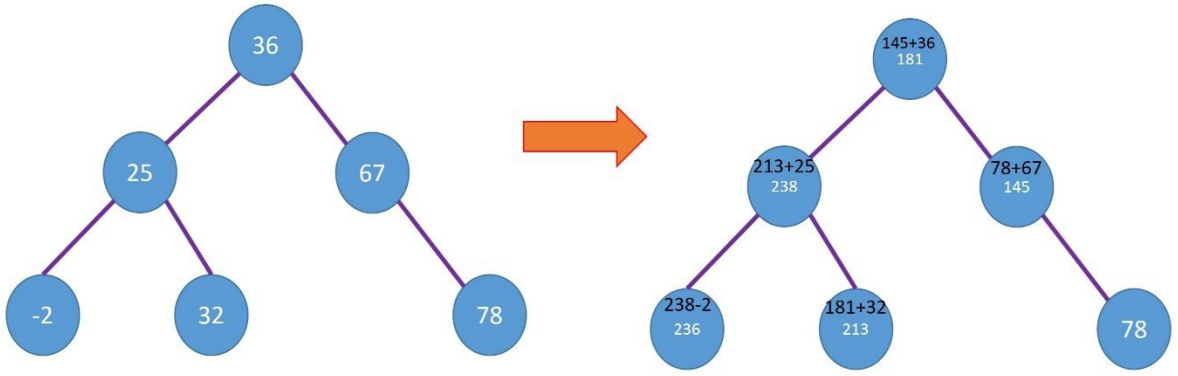


Figure 3: Original BST (Left) and Transform BST (right)

Sample input
36 25 67 -2 32 78

Sample input
36
|---25
| |----2
| |___32
|___67
| |___78

Sample output

181
|---238
| |---236
| |___213
|___145
| |___78

3. Given a Binary Search Tree (BST), implement a function to find and return the sum of all non-leaf nodes. A non-leaf node is any node that has at least one child.

Constraints:

- The BST is **non-empty**.
- The BST follows the standard **binary search tree** property:
  - The left subtree contains only nodes with values **less than** the root.
  - The right subtree contains only nodes with values **greater than** the root.
- The function must use **recursion** to compute the sum of non-leaf nodes.

Function Prototype:

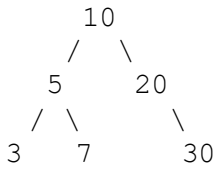
```
def sumOfNonLeafNodes(root):
```

Input Format:

1. The BST is built from user input.
2. The user enters **multiple integers**, and each integer is inserted into the BST.
3. The function `sumOfNonLeafNodes(root)` is called to compute the sum.

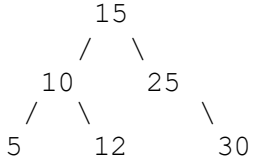
Output Format:
An **integer** representing the sum of all non-leaf nodes.

Example 1:
Input (BST Structure):



Output
Sum of non-leaf nodes: `10 + 5 + 20 = 35`

Example 2:
Input (BST Structure):



Output
Sum of non-leaf nodes: `15 + 10 + 25 = 50`

5. **(postOrderIterativeS2)** Write an iterative Python function `postOrderIterativeS2()` that prints the post-order traversal of a binary search tree using **two stacks**. Note that you should **only** use `push()` or `pop()` operations when you add or remove integers from the stacks. Remember to empty the stacks at the beginning, if the stacks are not empty.

The function prototype is given as follows:

```
def postOrderIterativeS2(root):
```

Let us consider the below tree for example

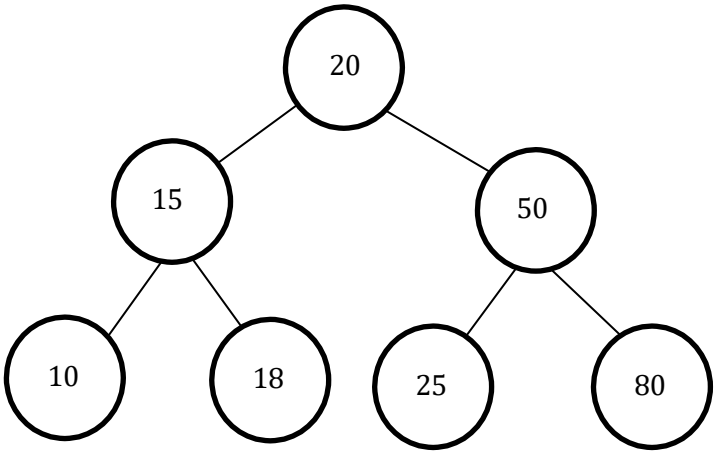


Figure 4: Iterative Postorder Tree Traversal: 10 18 15 25 80 50 20